

# Basic Walkthrough Reverse Engineer in CTF

By Sala Cyber

## Introduction

This is a walkthrough document we prepared to help you get started with reverse engineering in Capture the Flag (CTF) challenges. It covers key concepts, tools, and techniques commonly used in CTF reverse engineering tasks, and includes step-by-step examples to guide your learning.

## Demo example challenge name: Stack

In this stack-based challenge, the goal is to uncover a password that is stored within the stack memory. Solving it requires applying multiple reverse engineering techniques, including stack analysis, disassembly inspection, and runtime debugging.



stack.exe

```
Command Prompt
Microsoft Windows [Version 10.0.26100.4351]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>C:\Users\User\Desktop\Jica_CTF_Training\demo\stack.exe
help:
C:\Users\User\Desktop\Jica_CTF_Training\demo\stack.exe <password>

C:\Users\User>C:\Users\User\Desktop\Jica_CTF_Training\demo\stack.exe 12345
[!] InCorrect Password

C:\Users\User>
```

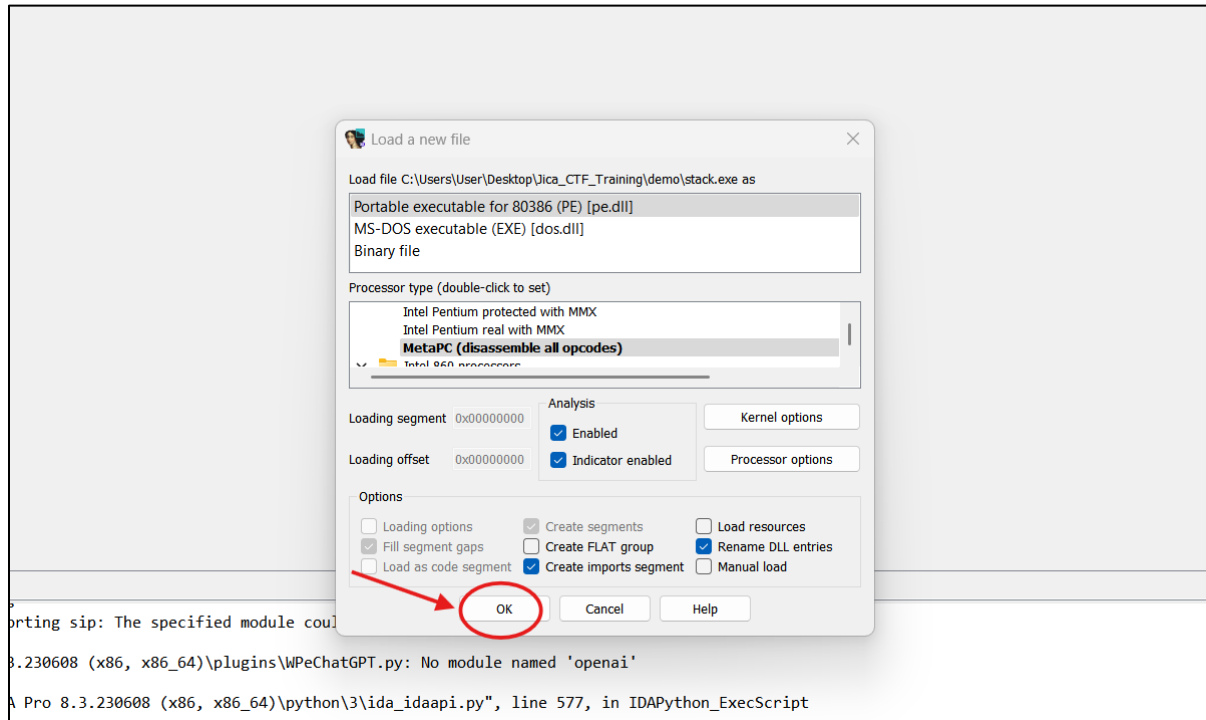
### List of tools:

- IDA (<https://hex-rays.com/ida-free>) or (<https://hex-rays.com/ida-pro>)
- Ghidra (<https://github.com/NationalSecurityAgency/ghidra>)
- X64dbg/x86dbg (<https://x64dbg.com/>)

**Recommendation:** All these tools should only be used in a sandboxed environment such as a virtual machine designed for analyzing unknown binary, do not attempt to analyze unknown binary on your host operating system.

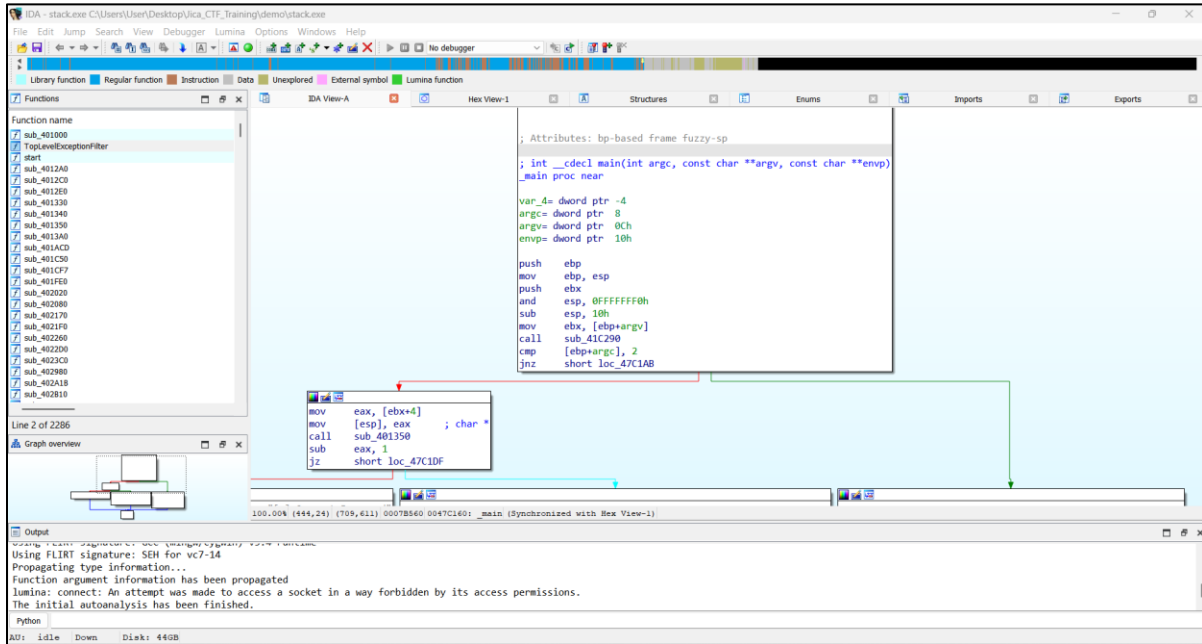
## For basic static analysis (using IDA)

Step 1 load or drop the file into IDA

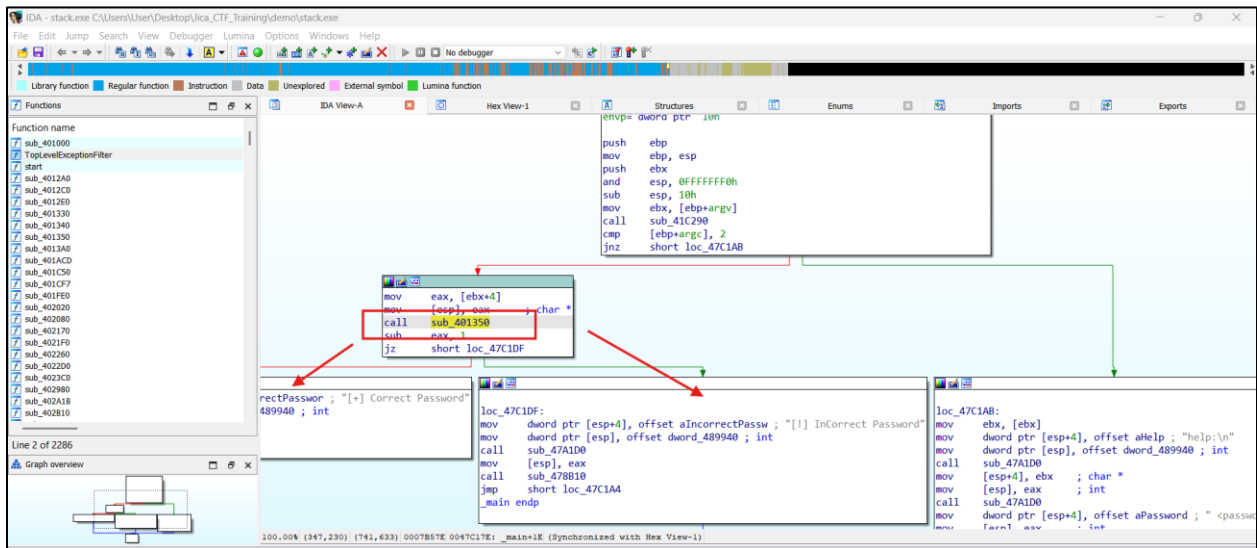


Then click on “OK”.

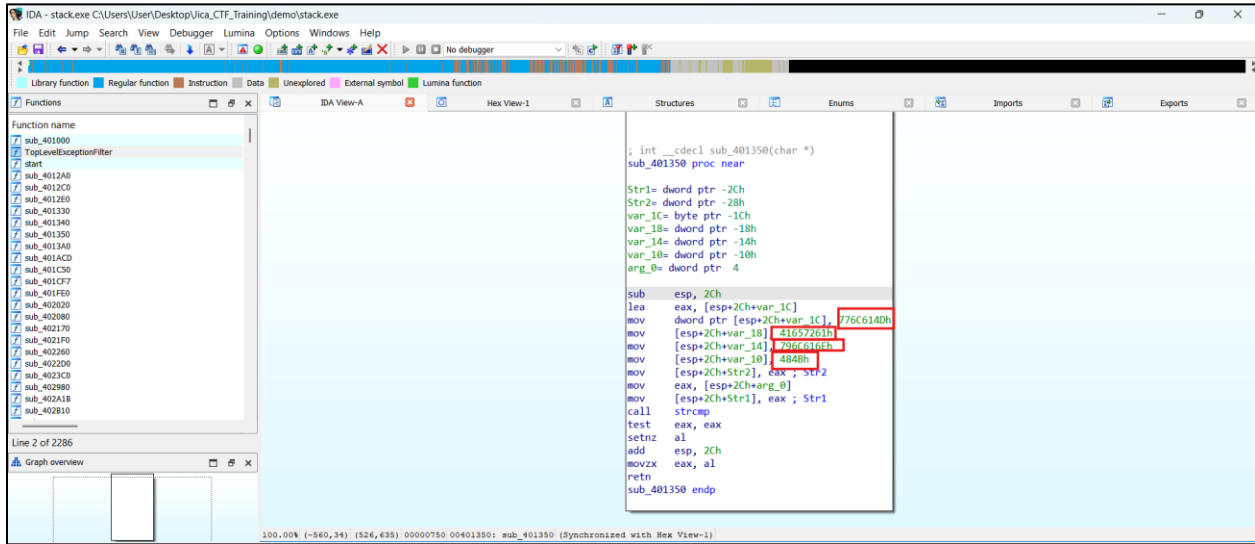
Step 2: waiting until the analysis from IDA is done and showing the graph view like this blow picture:



Step 3: Locate the calling function that has two branching paths. And go to the function.



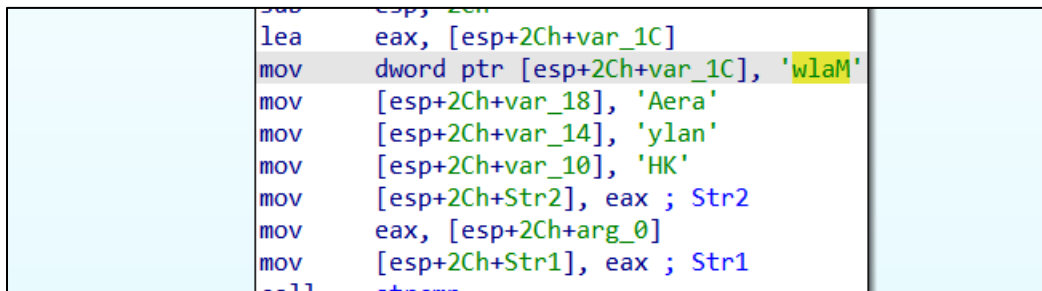
Step 4: In the function `sub_401350`, we can see hex bytes that can be converted to strings. By selecting the hex and then right click > select 'x' F2



```

; int __cdecl sub_401350(char *)
sub_401350 proc near
    Str1= dword ptr -2Ch
    Str2= dword ptr -28h
    var_1C= byte ptr -1Ch
    var_18= dword ptr -18h
    var_14= dword ptr -14h
    var_10= dword ptr -10h
    arg_0= dword ptr 4

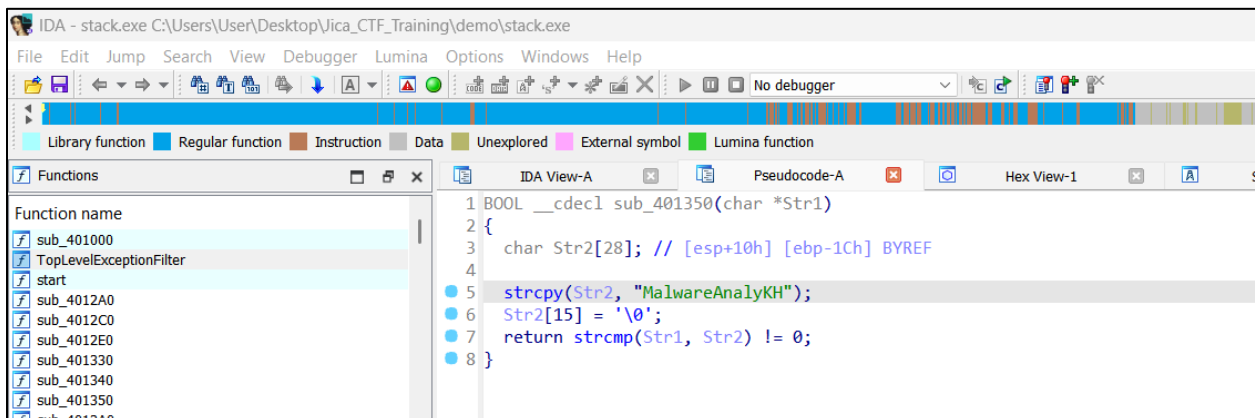
    sub     esp, 2Ch
    lea     eax, [esp+2Ch+var_1C]
    mov     dword ptr [esp+2Ch+var_1C], 776C6140h
    mov     [esp+2Ch+var_18], 72864816h
    mov     [esp+2Ch+var_10], 4B4Bh
    mov     [esp+2Ch+Str2], eax ; Str2
    mov     eax, [esp+2Ch+arg_0]
    mov     [esp+2Ch+Str1], eax ; Str1
    call    strcmp
    test    eax, eax
    setnz   al
    add     esp, 2Ch
    movzx   eax, al
    retn
sub_401350 endp
  
```



```

lea     eax, [esp+2Ch+var_1C]
mov     dword ptr [esp+2Ch+var_1C], 'wlaM'
mov     [esp+2Ch+var_18], 'Aera'
mov     [esp+2Ch+var_14], 'ylan'
mov     [esp+2Ch+var_10], 'HK'
mov     [esp+2Ch+Str2], eax ; Str2
mov     eax, [esp+2Ch+arg_0]
mov     [esp+2Ch+Str1], eax ; Str1
call    strcmp
  
```

Also, if we have access to decompiler we can go to **view > subview > generate pseudocode** (on clicking F5 key)



```

1 BOOL __cdecl sub_401350(char *Str1)
2 {
3     char Str2[28]; // [esp+10h] [ebp-1Ch] BYREF
4
5     strcpy(Str2, "MalwareAnalyKH");
6     Str2[15] = '\0';
7     return strcmp(Str1, Str2) != 0;
8 }
  
```

Flag = **MalwareAnalyKH**

```
Command Prompt
Microsoft Windows [Version 10.0.26100.4351]
(c) Microsoft Corporation. All rights reserved.

C:\Users\User>C:\Users\User\Desktop\Jica_CTF_Training\demo\stack.exe
help:
C:\Users\User\Desktop\Jica_CTF_Training\demo\stack.exe <password>

C:\Users\User>C:\Users\User\Desktop\Jica_CTF_Training\demo\stack.exe 12345
[!] InCorrect Password

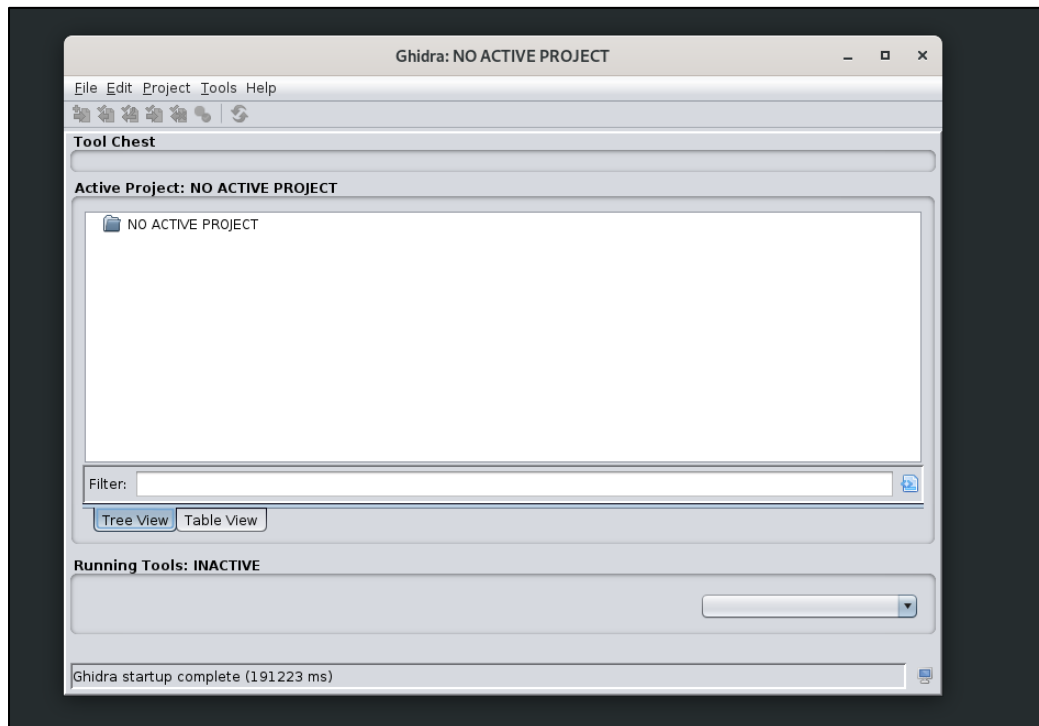
C:\Users\User>C:\Users\User\Desktop\Jica_CTF_Training\demo\stack.exe MalwareAnalyKH
[+] Correct Password

C:\Users\User>
```

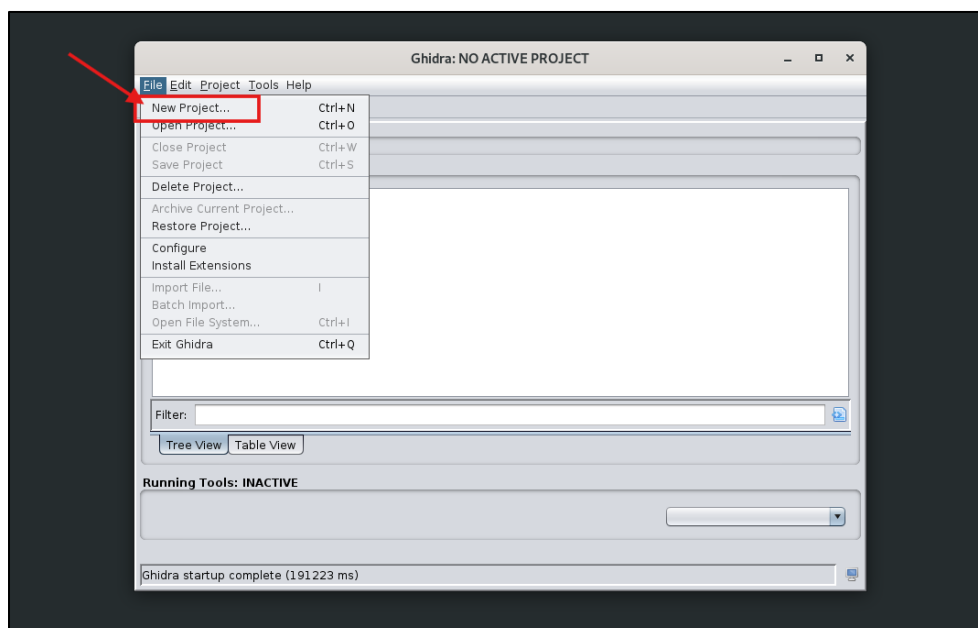
For basic static analysis (using Ghidra)

## How to get started with Ghidra

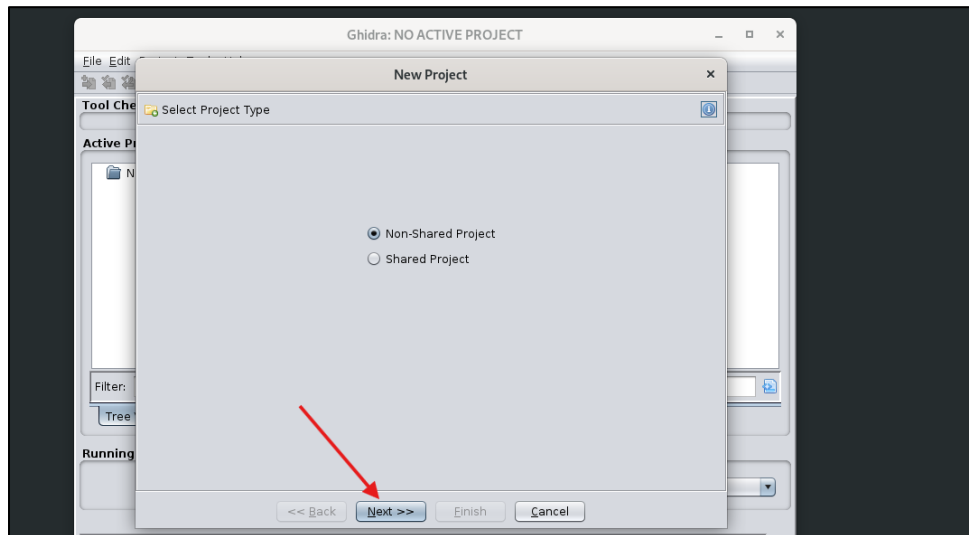
To begin using Ghidra to analyze a sample you first need to create a project which is used to store the sample and any files generated by Ghidra.



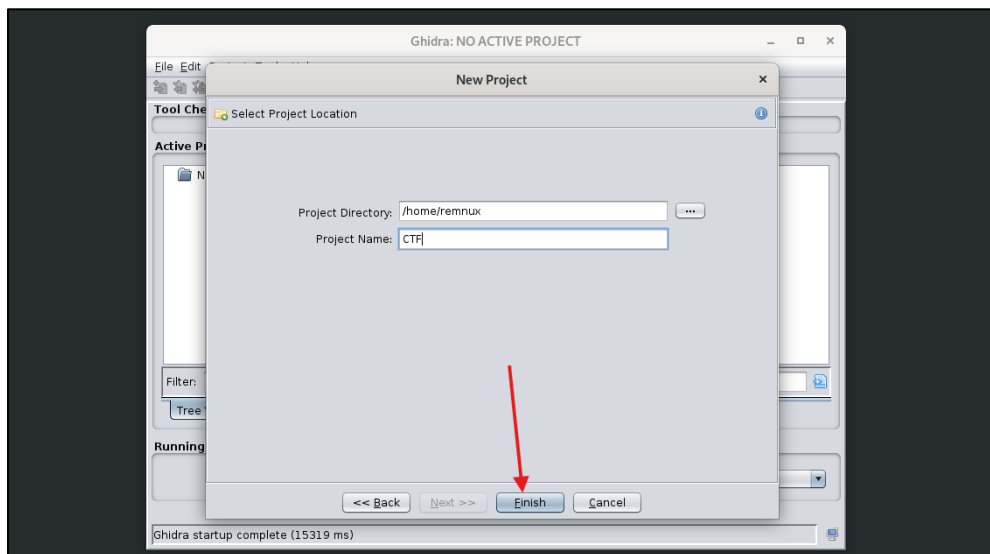
To begin, select 'File', then 'New Project'.



Select whether you want to share the project or not, in this example, I will choose 'Non-Shared Project' and click 'Next'.



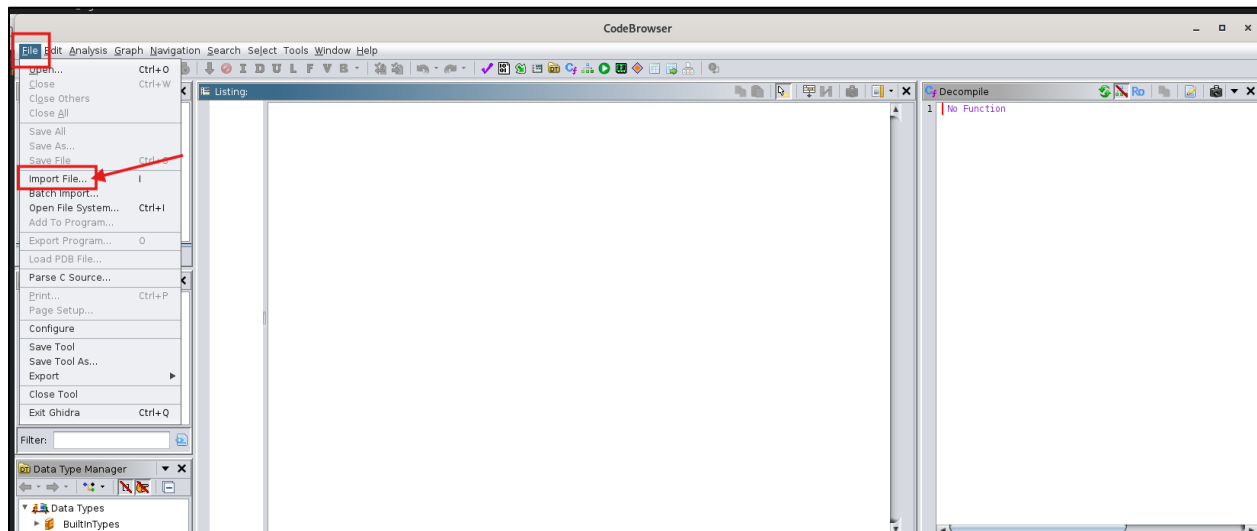
Input name as you prefer (CTF) in this example, next click on finish.



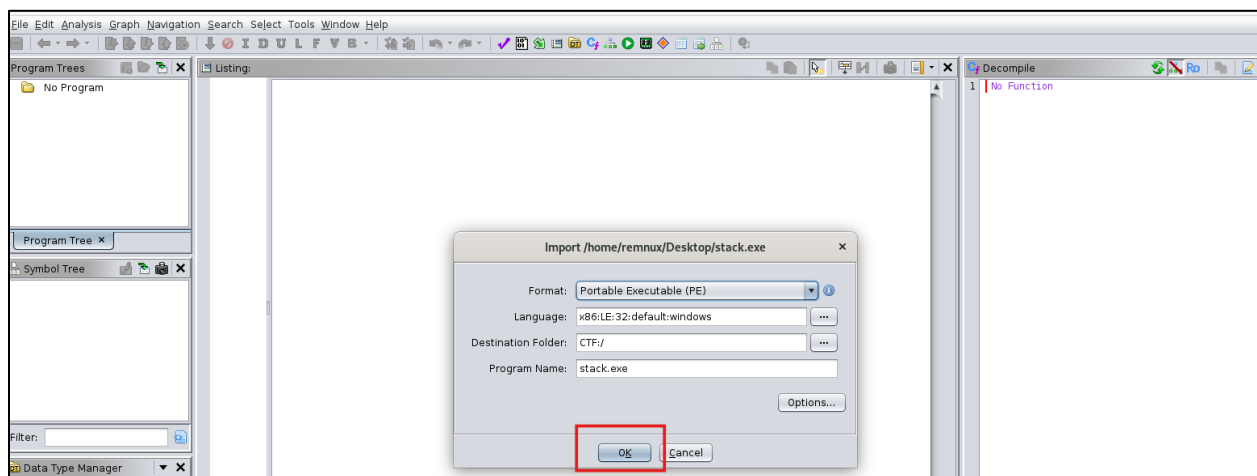
We can start by clicking on the dragon head logo



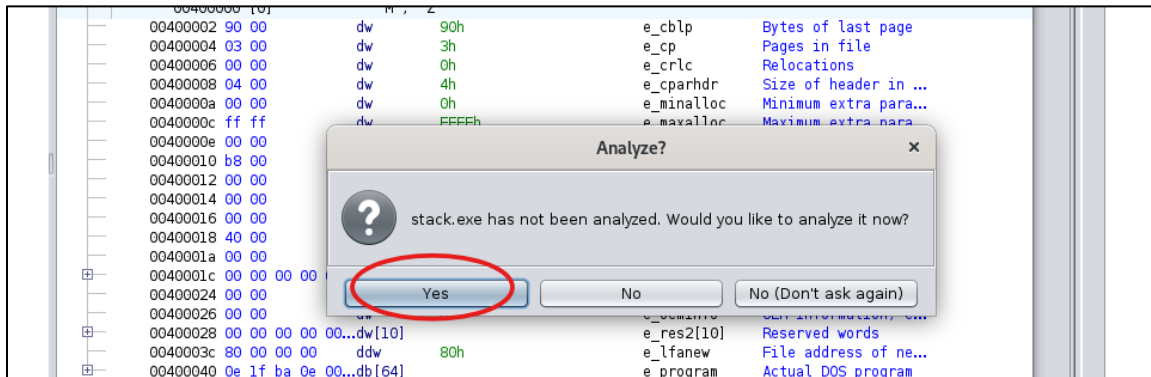
Step 1: import file by go to **File > Import File**



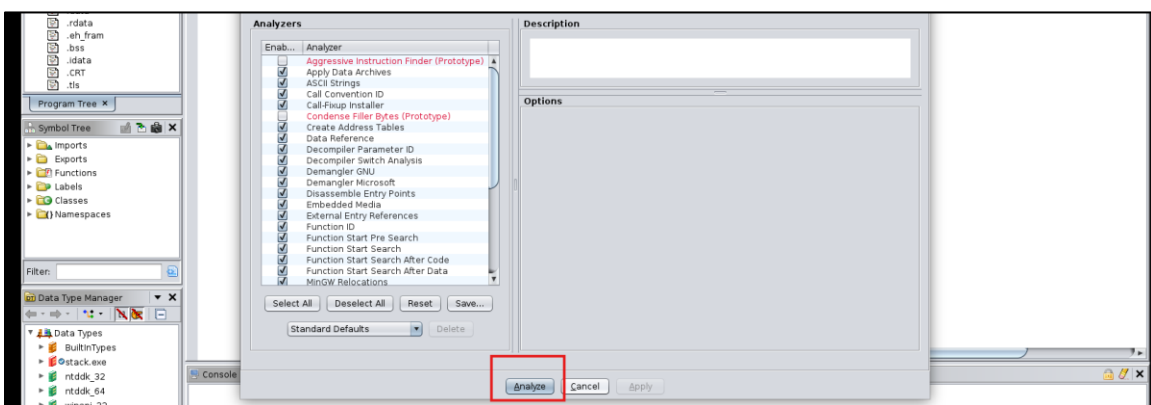
Step 2: click on OK





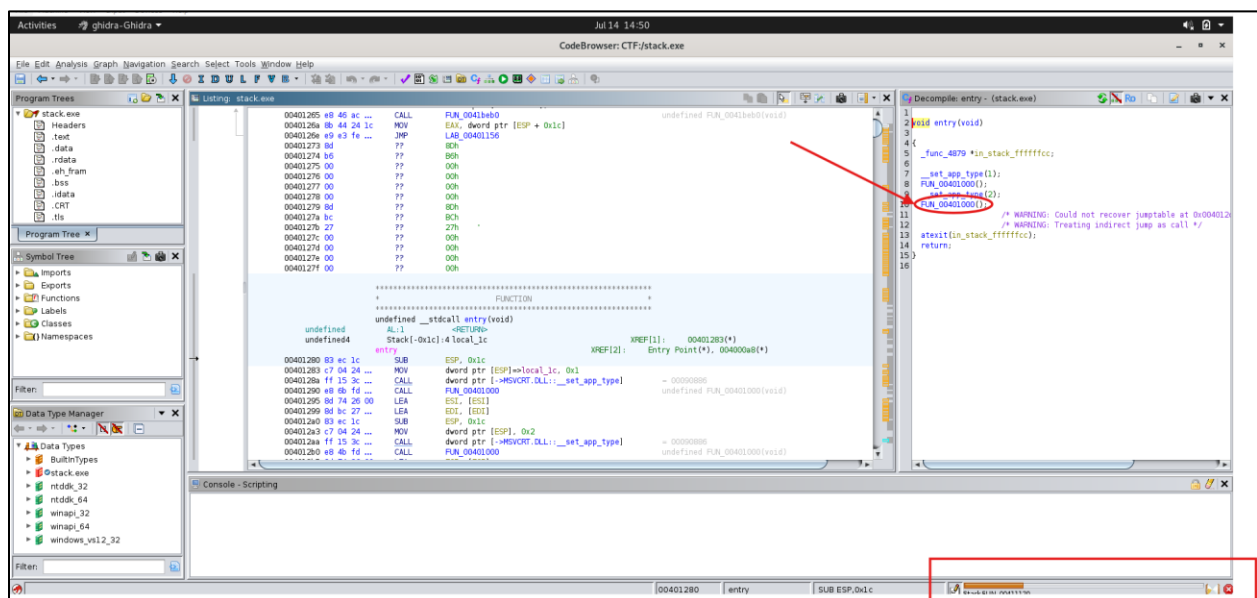


Click on Yes.



And then click on Analyze.

Step 3: Please wait until the end of analyzing done first and we can click go on (Fun\_0040100)

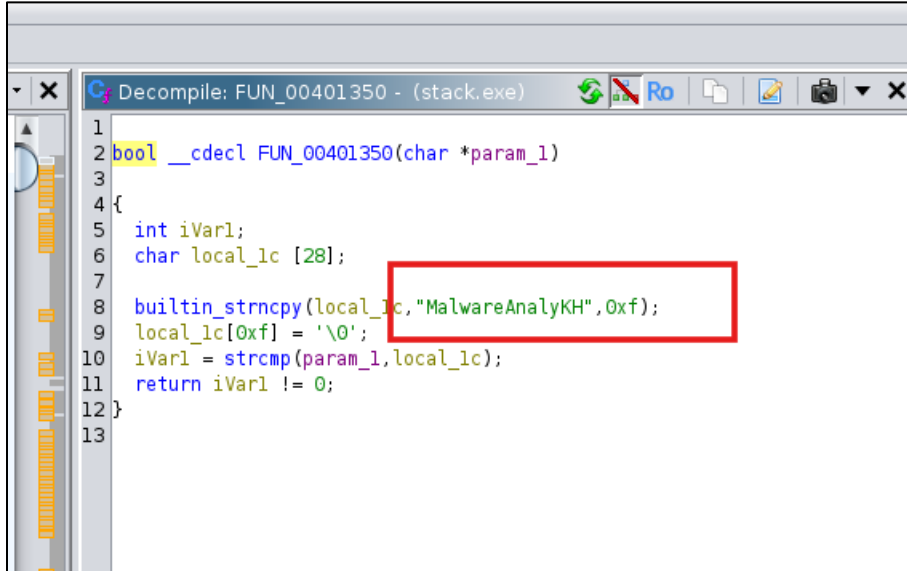


```

1
2 undefined4 __cdecl FUN_0047c160(int param_1,undefined4 *param_2)
3
4 {
5     char *pcVar1;
6     bool bVar2;
7     undefined3 extraout_var;
8     int *piVar3;
9
10    FUN_0041c290();
11    if (param_1 == 2) {
12        bVar2 = FUN_00401350((char *)param_2[1]);
13        if (CONCAT31(extraout_var,bVar2) == 1) {
14            piVar3 = FUN_0047ald0((int *)&DAT_00489940,"[!] InCorrect Passwor
15            FUN_00478b10(piVar3);
16        }
17        else {
18            piVar3 = FUN_0047ald0((int *)&DAT_00489940,"[+] Correct Password"
19            FUN_00478b10(piVar3);
20        }
21    }
22    else {
23        pcVar1 = (char *)*param_2;
24        piVar3 = FUN_0047ald0((int *)&DAT_00489940,"help:\n");
25        piVar3 = FUN_0047ald0(piVar3,pcVar1);
26        FUN_0047ald0(piVar3," <password>\n");
27    }
28    return 0;
29 }
30

```

Page 10 of 11



```
1
2 bool __cdecl FUN_00401350(char *param_1)
3
4 {
5     int iVar1;
6     char local_1c [28];
7
8     builtin_strncpy(local_1c,"MalwareAnalyKH",0xf);
9     local_1c[0xf] = '\0';
10    iVar1 = strcmp(param_1,local_1c);
11    return iVar1 != 0;
12 }
13
```

Step 6: We can see the flag is (**MalwareAnalyKH**)